ARTCRAFT FLEET TECHNICAL BROCHURE · SERIES #25

GainCraft v0.1.0

Customer health for a small B2B business: usage aggregates per account and day, contract data from contractcraft, unhappy-customer signals from deskcraft, an explainable 0-100 health score, and calls to action (CTAs) that say what to do this week. One SQLite file, one binary that is both CLI and MCP server. Part of the back-office craft suite (replaces Gainsight / ChurnZero / Totango / a "customer health" spreadsheet).



v0.1.0	3 Crates	3510	6-Gate
RELEASE VERSION	WORKSPACE TIERS	PURE-RUST LOC	ARTCRAFT VERIFIED

Repository: `D:\Dev\repos\gaincraft` · **Standard:** `CRAFT_BROCHURE_STANDARD.md` · **Compiled via:** `BookCraft v0.5.0`

Customer health for a small B2B business: usage aggregates per account and day, contract data from contractcraft, unhappy-customer signals from deskcraft, an explainable 0-100 health score, and calls to action (CTAs) that say what to do this week. One SQLite file, one binary that is both CLI and MCP server. Part of the back-office craft suite (replaces Gainsight / ChurnZero / Totango / a "customer health" spreadsheet).

Status: **v0.1.0, Tier 1** (store + CLI + MCP, tested). No desktop window yet. Synthetic demo data only. **No per-person tracking**: gaincraft only stores counts per account and day and refuses import files with user, e-mail, name, login or IP columns.

- How to use it day to day: HELP.md
- What it is for and where it stops: docs/PRD.md
- Every MCP tool: docs/TOOLS.md (generated, checked by a test)
- Changes: CHANGELOG.md

What it does

- **Accounts** with key, domain (links mail signals), owner, ARR, seats, contract number, term end, notice date. Found by key, domain or name.
- **Telemetry import:** CSV `account;date;active_users[;sessions;key_actions]` (`;` or `,`). Every row is validated and every account resolved before anything is written; re-importing a day replaces it. Per-person columns are refused before a single row is read (ArbVG section 96a, DSGVO data minimisation).
- **Health score**, deterministic and shown with its parts:

```
score = 100 x (0.45 adoption + 0.35 recency + 0.20 sentiment) x modifier
```

- adoption: average active users over 30 days / seats (or / best day in 90 days without seats), capped at 1
- recency: halves every 14 days without activity
- sentiment: 1 minus decayed negative signals (30-day half-life) plus positive ones, over 180 days
- modifier: optional advisory 0.8-1.2 from an LLM or human read, with a reason and an expiry
- bands: green >= 70, yellow >= 40, red below. Weights are configurable and must sum to 1.
- **CTAs**, opened once per account, kind and reference: `low_health`, `adoption_drop` (last 14 days below 60% of the 14 before), `seats_unused` (under half the seats), `renewal` (notice date within 60 days), `negative_signal` (last 30 days). Resolving needs a note.
- **Bus inbox:** `contract.executed` from contractcraft sets ARR, contract number, term end and notice date (matched by contract number, then counterparty name); `signal.negative` from deskcraft (and later formcraft) becomes a sentiment signal, matched by mail domain. The same ticket counts once even if it is sent again. Unknown accounts are created and flagged `auto_created`.
- **Deadlines:** notice dates and term ends within N days, so renewals are not missed.

Quick start

```
gaincraft demo-seed
gaincraft health-all
gaincraft show mockinger.example
gaincraft cta
gaincraft deadlines --days 120
```

With the other suite apps (same bus, `%LOCALAPPDATA%\craft-biz\bus` or `$CRAFT_BIZ_BUS`):

```
deskcraft signals --emit
contractcraft move C-2026-003 executed     # after sealing it
gaincraft inbox-drain
```

Your own data: `gaincraft add "Kunde GmbH" --domain kunde.at --seats 12 --arr "14.400,00"`, then export daily active users per customer from your product (a SQL `GROUP BY customer, day`) and `gaincraft import usage.csv`.

Database: `--db PATH`, else `$GAINCRAFT_DB`, else `%LOCALAPPDATA%\gaincraft\gaincraft.db`.

MCP

```
{ "mcpServers": { "gaincraft": { "command": "D:/Dev/cargo-
target/release/gaincraft.exe", "args": ["mcp", "--allow-path", "D:/Exports/usage"] } }
}
```

16 tools, all `gain_*`; see docs/TOOLS.md. An agent can read an account and set an advisory triage modifier with a reason; it cannot move the score outside 0.8-1.2 of the deterministic value. Agent actions are recorded with actor `agent`. Without `--allow-path` the server cannot read any file.

Build

```
just ci           # clippy -D warnings, tests (incl. docs/TOOLS.md sync), fmt check
just docs         # regenerate docs/TOOLS.md after changing a tool
just build-release
```

Needs `../craft-biz` checked out next to this repo.

Not done (honest list)

See the "Gaps" section of docs/PRD.md. In short: no window, no connectors to product databases (CSV only), the weights are a sensible default and not fitted to churn data, no NPS or survey intake until formcraft exists.

Product Requirements & Architecture Specification

What it is

A customer-health tool for small B2B businesses (5-200 customers): one place that combines how much each customer uses the product, what their contract says, and whether they are unhappy, turns that into a score you can explain in one sentence, and lists what to do about it. Local, one SQLite file, CLI + MCP. Replaces Gainsight, ChurnZero, Totango and the health spreadsheet nobody updates.

Users

- The owner or account manager of a small software or service business who is responsible for renewals.
- An agent (via MCP) that drains events, recomputes health and reads tickets to propose a triage modifier.

Jobs

1. Know which customers are drifting away before the notice date passes.
2. See why a score is what it is (adoption, recency, sentiment, modifier), not just a traffic light.
3. Get a short, deduplicated list of things to do, and record what was done.
4. Pull in contract dates and support mood from the other craft apps without retyping.
5. Do all of this without building per-employee surveillance of the customer's staff.

Today (v0.1.0)

- Accounts with ARR, seats, contract number, term end, notice date; lookup by key, domain or name.
- Aggregate telemetry import (CSV), all-or-nothing, idempotent per day, refuses per-person columns.
- Deterministic health score with configurable weights, stored snapshots, explainable components.
- Advisory triage modifier (0.8-1.2, reason, expiry), enforced in Rust and by a SQLite CHECK.
- Five CTA rules, opened once per account/kind/reference, resolution needs a note.
- Bus inbox: `contract.executed` (contractcraft) and `signal.negative` (deskcraft, formcraft later); one signal per source document; unknown accounts created and flagged.
- Deadlines view for notice dates and term ends.
- 16 MCP tools, CLI with tables and a health breakdown, demo seed matching the contractcraft and deskcraft demos.
- 22 tests; live chain verified with the real binaries: deskcraft signal -> gaincraft sentiment; contractcraft seal + execute -> gaincraft ARR, contract, notice date.

Boundaries

- No per-person data, ever. Aggregates per account and day only. This is a design decision (ArbVG section 96a, DSGVO), not a missing feature.
- No outbound messages. gaincraft says who to call; it does not send campaigns.
- No ML churn prediction. The score is a transparent formula; an LLM may only nudge it within 0.8-1.2 with a written reason.
- No CRM pipeline or invoicing; contracts live in contractcraft, invoices in buchcraft.

Gaps (honest)

- No desktop window (Tier 2).
- CSV only: no direct connection to a product database or analytics tool.
- The default weights (0.45 / 0.35 / 0.20) and the 14-day and 30-day half-lives are reasoned defaults, not fitted to anyone's churn history.
- `seats_unused` and `adoption_drop` are simple thresholds; seasonal businesses will see false alarms (dismiss with a note).
- No NPS or survey intake yet; formcraft will send `signal.negative` (and later positive) responses.
- Contract changes after execution (amendments, terminations) are not sent by contractcraft yet, so ARR and dates can go stale; edit with `update`.
- A CTA that is resolved stays closed even if the same condition persists (by design, to avoid nagging); a new reference reopens.

Next

1. formcraft: survey responses as signals (negative and positive).
2. contractcraft: `contract.terminated` / `contract.amended` events.
3. Tier 2 window: account list with sparkline trend, CTA board.
4. Optional backtest: given past churned accounts, report how the score ranked them before they left (to tune weights honestly).

Complete MCP Tool & CLI Verb Reference

Generated by `gaincraft tools --markdown`. Do not edit by hand: a test fails when this file is out of date. Every tool is also a CLI verb (`gaincraft --help`). Dotted names (`app.verb`) are accepted.

Tool	What it does	Required	Optional
`gain_status`	Active accounts, ARR, telemetry days, open CTAs, signals in the last 90 days, health weights.	-	`as_of`
`gain_account_add`	Add a customer account. Domain links signals from deskcraft (mail domain).	`name`	`arr`, `domain`, `key`, `notes`, `owner`, `seats`
`gain_account_list`	Accounts with ARR, seats, contract dates.	-	`include_inactive`
`gain_account_show`	One account with current health and components, score trend, signals, open CTAs and triage.	`account`	`as_of`
`gain_account_update`	Change account fields; null clears. active=false retires an account.	`account`	`active`, `arr`, `contract_id`, `domain`, `last_notice`, `name`, `notes`, `owner`, `renewal_months`, `seats`, `term_end`
`gain_telemetry_import`	Import usage aggregates per account and day: account;date;active_users[;sessions;key_actions]. Per-person columns are refused (ArbVG / DSGVO). Re-importing a day replaces it.	`path`	`dry_run`
`gain_signal_add`	Record a customer signal by hand (e.g. after a call): negative or positive, score 1-10, with a note.	`account`, `score`, `note`	`date`, `polarity`, `reference`
`gain_health`	Health score 0-100 of one account with components (adoption, recency, sentiment, modifier); stores a snapshot and raises CTAs.	`account`	`as_of`
`gain_health_all`	Health of all active accounts, worst first, ARR at risk (yellow + red); stores snapshots and raises CTAs.	-	`as_of`
`gain_triage_set`	Set an advisory modifier (0.8-1.2, with a reason, expires after N days) from an LLM or human read of the account; clear=true removes it.	`account`	`clear`, `days`, `modifier`, `reason`
`gain_cta_list`	Open calls to action (low health, adoption drop, unused seats, renewal, negative signal), due first.	-	`account`, `include_closed`

Tool	What it does	Required	Optional
`gain_cta_resolve`	Resolve (or dismiss) a CTA with a note saying what was done.	`id`, `note`	`dismiss`
`gain_deadlines`	Notice deadlines and term ends of accounts within N days (default 90).	-	`as_of`, `within_days`
`gain_weights`	Show or set the health weights (adoption, recency, sentiment; must sum to 1).	-	`set`
`gain_inbox_drain`	Apply events from other craft apps: contractcraft contract.executed (ARR, contract dates) and deskcraft/formcraft signal.negative (sentiment). Unknown accounts are created and flagged.	-	-
`gain_demo_seed`	Fill an EMPTY database with three labelled demo accounts (healthy, dropping, fading) whose names match the other craft demos.	-	-